3 Literature Review

3.1 Introduction

This literature review aims to provide an understanding of manufacturing processes and the problem of organising them, how optimisation has been done before, machine learning and it's different categories, and how ML has been used in optimisation. The Job Shop Scheduling Problem is a challenge all operations need to provide a solution for in order to know what jobs to work on and when. However, current solutions are either costly and time consuming, or they have diminishing returns and are short sighted. ML is great at gathering large amounts of data and recognizing patterns. Having already been applied in the manufacturing industry including in research to improve optimisations algorithms such as particle swarm optimisation there remains opportunities to find patterns involved in the JSSP previously missed.
Saunders Research Onion was used to identify the research and development methods most applicable for this project to be effective. Abductive research was used for the literature review as I already had an understanding of the field to go back and forth between the literature 2 review and software to simultaneously test my knowledge and ideas, while still learning, for faster and more in-depth research and development.

3.2 Manufacturing Processes

3.2.1 Job Shop Scheduling Problem

The Job Shop Scheduling Problem (JSSP) is a complex theoretical model where a set of N jobs must be processed across M machines. Each job corresponds to an individual part and consists of a specific sequence of operations that must be executed in a unique, predetermined order. The amount of resources that need to be considered increases rapidly when solving this problem in real-world applications. Manufacturers aren't just considering a single goal

but rather multiple aims and objectives during production.

Simple rules to decide what jobs should be processed on which machines and when would be easy to apply but don't consider all the information and objectives of the environment and operators. In addition a production environment is dynamic and stochastic by nature while simple rules are static and deterministic. The dynamic nature of a production environment needs to be considered. Decisions on what actions should be taken will take time, so the methods used to make such decisions should be considered in terms of how long they will take because the production environment may have changed during that time (Subramaniam n.d.).

Real-world production systems have to be simplified to create a process that can find a solution for the JSSP. This requires assumptions to be made and as such, different assumptions and considerations are used for respective implementations of the JSSP. While are variations, the assumptions of the what is known as the classical JSSP are:

- All jobs are available at time zero: Every job that needs to be scheduled is completely independent and ready to be processed at the very outset of the schedule.
- Continuous machine availability: Every resource or machine is assumed to be continuously operational. Unpredictable events like machine breakdowns or maintenance delays are entirely neglected.
- No pre-emption is allowed: Once an operation begins processing on a machine, it must run to completion without any interruption. The processing cannot be paused or split.
- Deterministic parameters: The exact required processing times for every operation and their predefined route (precedence constraints) are completely known in advance.
- Strict capacity constraints: Each machine can process only one operation at a time, and a single job can only be processed on one machine at a time.
- Simplified setup and travel times: The time required to set up a machine is assumed to be either negligible, sequence-independent, or already factored into the known processing time of that operation. Travel times between stages are also not considered.
- Infinite buffers: In-process inventory is allowed, meaning if the next machine a job needs is busy, the job can wait in a queue indefinitely.

- Non-delay processing: A resource is never deliberately kept idle if there is a pending operation ready to be processed.

Variations

No-wait Job Shop Scheduling Problem refers to there being no time in between a job being processed by each machine in its sequence. This is formally described as having a maximum time of zero the same assumptions as the classical JSSP (Grimes & Hebrard 2011). **The Flexible Job Shop Scheduling Problem (FJSP)** allows for multiple possible routes of production. This is done by having several machines running in parallel that can all perform the same work at a point in a job's route. Gao et al. (2023) states that the assumptions of the FJSP are:

- All machines are idle at the start of the scheduling process.
- All jobs can be processed at the start of the scheduling process.
- All jobs have the same priority.
- There are no constraints on the order of operations between different jobs.
- Each processing operation cannot be interrupted.
- There are no limits to the buffer area for each machine to store jobs.

Dynamic Scheduling Problems

3.2.2 Goals of Optimisation

To effectively optimise any process no matter what the methods it is important to know what the aim of optimisation is. In addition, identifying which goals should be prioritized is important for the best performance for the chosen domain. Most manufacturers are profit driven and as such aim to reduce costs and increase revenue. Looking at the key performance indicators (KPIs) of large companies gives an indication what the goals of optimisation should be. Ahmad & Dhafr (2002) defines a KPI as "a number or value which can be compared against an internal target, or an external target 'benchmarking' to give an indication of performance". Looking at annual reports from 2024 and 2025 of 5 major engineering companies the main

KPIs talked about are: revenue/sales, profits, return on investment, injury rates, and green house gas emissions (*Annual Financial Report 2024* n.d.).

The ISO 55000 standards give companies and asset manages a framework to identify their KPIs and structure how they fit into the wider world. This is done by working with stakeholders to create aims and objectives of an organisation that give it purpose as will as the constraints. Leadership is used to ensure there are individuals responsible and accountable for decisions being made and that they are clearly communicated. Planning is required in the standard to define prioritisation methods, allocate resources, consider asset lifecycle, evaluate results, and consider risks involved. Collaboration with other groups both inside and out of the organisation is recommended. The performance of assets and the use of indicators should be used to measure the achievement of goals. This can then help in reviewing past failures identifying potential ones in the future to improve and mitigate.

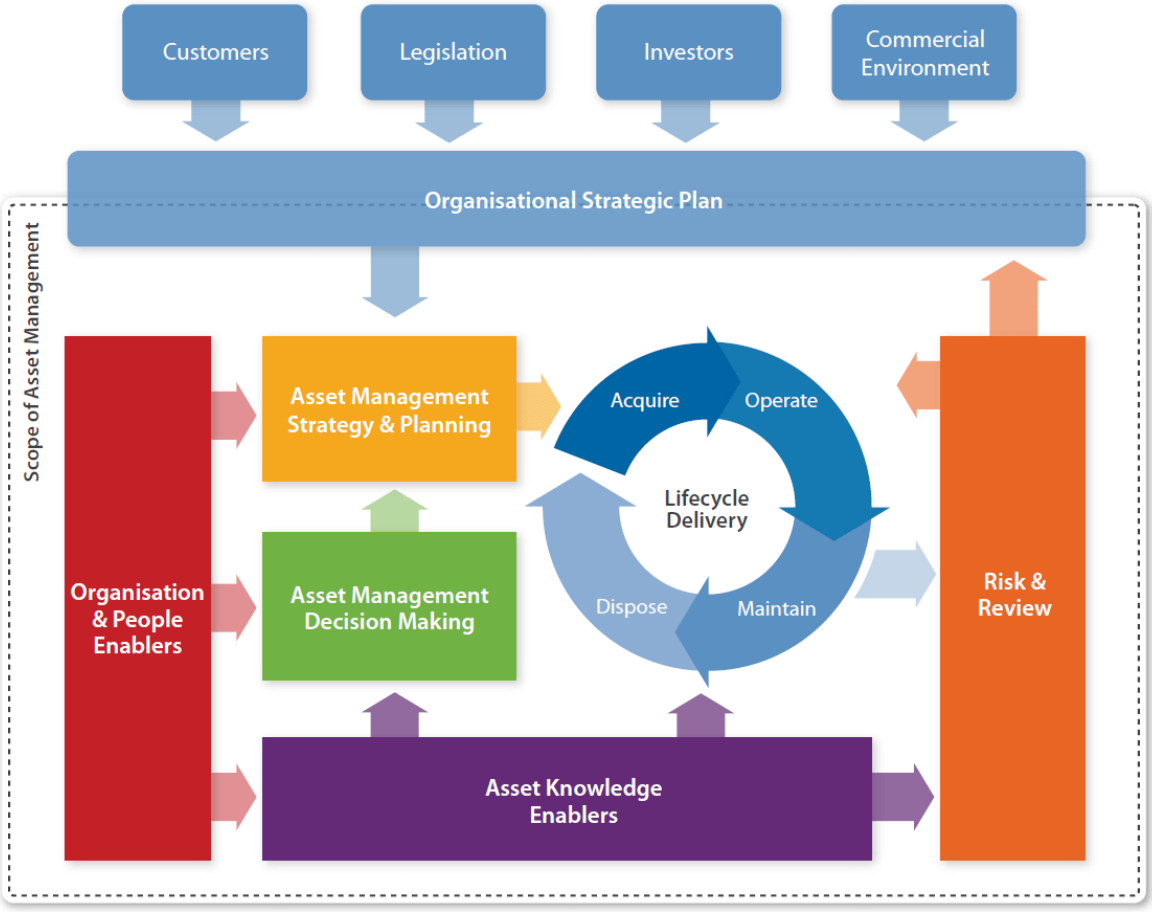


Figure 3.1: Institute of Asset Management's (IAM's) Conceptual Asset Management Mode of ISO 55000

3.3 Conventional Optimisation Methods

This section discusses the capabilities and limitations of different heuristics and algorithms not including ML which are used for optimisation. These methods can be used to provide solutions for the JSSP in different ways. There are a variety of methods to solve the JSSP which work but given the scarcity of resources, better solutions are always being looked for. As such we need to search for a optimal solutions and identify processes that will produce them

Heuristics

The simplest forms of optimisation are heuristics such as: Shortest Processing Time (SPT), prioritise the job with the shortest time; Longest Processing Time (LPT), prioritising the job with the longest time (Alhijawi & Awajan 2024); Longest Processing Time (LPT), prioritizes the job with the longest processing time; Earliest Due Date (EDD), A static rule that prioritizes jobs based on their due date; First In First Out / First Come First Served (FIFO / FCFS), Processes jobs in the exact order they arrive; Minimum Slack Time (MST), Prioritizes jobs with the least amount of slack time (Ruiz & Maroto 2005). However, these a short sighted and only optimise for a single objective meaning they have limited use and aren't very effective on real factories. Ruiz & Maroto (2005) demonstrated the low performance of using these methods and pointed out their significant use in industry at the time which highlights the possibility to improve operations drastically with more effective methods. Further more, heuristics are fixed and as such can't accommodate flexible workshops which can use different possible routes or processes for production so inherently they won't always find the optimal solution as they can't consider different possibilities (Subramaniam n.d.)

Surface Response Methodology (RSM)

RSM was developed in the 1950s and provides optimal variables in the desired region of response. This region is where the output values which are being maximised or minimised co-coincide (Khuri & Mukhopadhyay 2010). This has the advantages of not needing many experiments and short cycles to optimise while also being very accurate (Song et al. 2024). However, modern computer software can determine the overlap of desired optimal values

quicker, easier and with more precision.

Genetic Algorithm (GA)

GAs work by taking an initial population of possible solutions to the task being optimised. From this population a set of candidates are chosen using a variety of techniques (e.g. Stochastic, roulette,). The every solution in the population will have a fitness function which compares how the solution performs to the average of the population. Solutions with a higher fitness function have a higher probability of being selected. Those selected with then have their processes combined with other selected solutions to form new processes. These new solutions then either completely replace the last population or a part of the last population is removed and the new solutions added in. This process is then repeated for a set amount of iterations or until another stopping criterion is met Alhijawi & Awajan 2024.

GAs are good at searching for a wide variety of solutions without getting stuck in local solutions and can be very adaptable to different optimisation problems. In addition, they can handle non-linear, non-convex optimisation problems, and dynamic environments.
The disadvantage of GAs is that it takes a long time to determine a solution and as such the parameters may need to be adjusted to improve the performance (Song et al. 2024). For frozen or slowly changing fields GAs work well but since they need to be re-optimised every time the plan is changed they do not work for dynamic environments such as when multiple orders are being placed over time and need scheduling quickly and reliably.

Particle Swarm Optimisation (PSO)

Random initial values are generated for the positions and velocity vectors of particles in a defined population size with each particle being given a fitness value based on this random initial position. Since this is the initial position it will considered the best fitness function for that particle and the next position is found using the initial velocity vector from the initial position. The best fitness function of all particles is known a the global best fitness function. To determine the next movement of each particle the vectors of velocity (inertia), from the particle position to the global best position, and from the current position to particle's best position. This is iterated to find the best position after a set amount of time given for optimisation. Weightings can be given to the different components deciding the new position

depending on which factor is desired to be favoured: inertia, individual best position, or global best position.

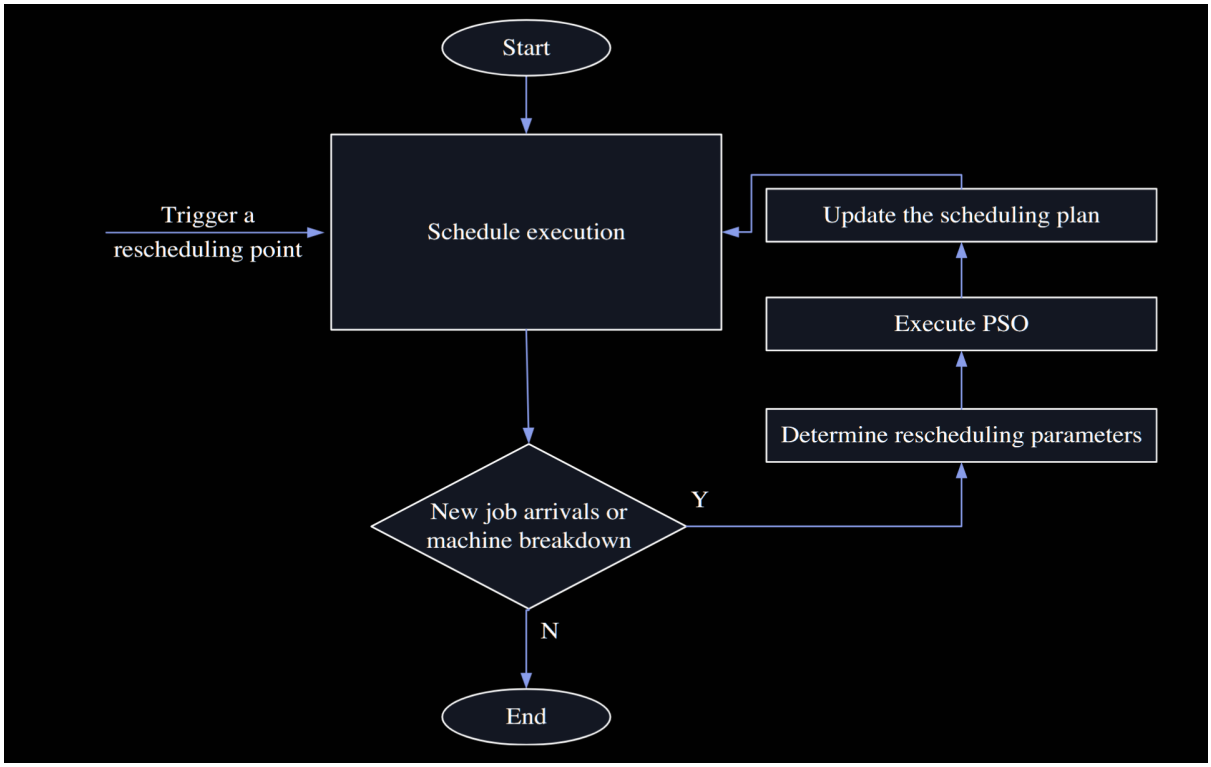


Figure 3.2: Flow of the dynamic solving process for the DFFS (Tang et al. 2016)

Figure 3.2 illustrates the rescheduling logic used in PSO when there's a new order, machine breakdown or other rescheduling trigger point.

Using particle swarm optimisation risks getting trapped in a local optimal solution in where the particles converge to one solution that appears to be optimal but there is actually a better solution that hasn't been explored. This can be mitigated by increasing the amount of particles used and time given for optimisation but this comes at the expense of being more computationally expensive (Tang et al. 2016).

Taguchi Method

Monte Carlo

Monte Carlo works by simulating many events to calculate an estimate of the probabilities different outcomes occur. This is useful for determining the outcome of complex systems that are determined from stochastic parameters. The first step is to define the distribution of the stochastic parameters using prior data. The model is then executed repeatedly, taking

random values for the necessary input parameters. The results of running the model after a given number of runs is then aggregated to determine the probabilities of different results.
Many simulations are necessary because the variability of results and their likelihood needs a large set of results to be accurate. In addition, the exploration of rare events requires covering a large amount of possible outcomes to find them. Often this requires 10s of thousands of simulations if not more to be effective.
Monte Carlo simulations are used in the pharmaceutical industry for drug discovery, simulating clinical trials and predict outcomes based on patients age, compliance, and disease progression. It's also used for production and risk management in industrial engineering, testing different scenarios to forecast and optimise based on variables such as market prices or variable manufacturing costs.
Estimation is heavily used and can limit the accuracy of results. In addition, millions of random variables may need generating and the reliability of their output is dependent on the quality and cycle period of the computer's Random Number Generator (RNG). Moreover, running so many simulations is time and computationally expensive which limits the flexibility of such an approach.

Limitations of Conventional Optimisation Methods

The main limitation facing conventional optimisation methods is the need for rescheduling when there is a state change. Furthermore, the static nature of these algorithms means they can't react to unexpected events such as a machine breakdown. There are strategies to provide short term reactive rescheduling but these are either limited or unrestricted which Tang et al. (2016) found can cause instability resulting in useless plans. The Job Shop Scheduling Problem is an NP hard problem where the number of states increases rapidly, meaning the computational cost gets increasingly higher and takes longer. This has resulted in approximation methods to be used which do not provide optimal solutions (Meeran & Morshed 2012). Large computational costs often result in only a single solution provided which removes human insight the opportunity to look into multiple possibilities to identify different possibilities (Wang & Shan 2007).

3.4 Machine Learning

Machine learning (ML) techniques have been developing rapidly in recent years and are being adopted across industries and disciplines. A key ability of ML is pattern recognition which is being researched and used for optimisation particularly for similar cases where at least partial repeatability is useful. While conventional optimisation techniques can often struggle with optimising for multiple objectives ML is able to manage this with different methods. Moreover, it is possible to handle quantitative as well as qualitative data which can then be used either as variables in the process or as objectives (Prestwich 2024).

3.4.1 Different ML Methods

ML for the most part comes under either supervised, unsupervised, and reinforcement learning. Together, these three methodologies serve as useful tools for developing effective and accurate solutions for complex data-driven decision-making problems (Ragab et al. 2019).

Supervised Learning

Supervised Learning (SL) is the most common ML technique and uses labelled datasets. The objective of this type of ML is to find the patterns between the inputs and outputs provided in the training data to create a function. The function takes in an input and produces an output and compares the produced output to the actual output. The difference is called the loss function and the aim is to minimise it during the model training. SL can be further split up into regression and classification categories based on the type of output variable (Taye 2023).

Regression is used for continuos data such as predicting house prices in a certain city based on size, location, and other provided variables (AlShaikh et al. 2025).

Classification is used for discrete or categorical data such as if an email is spam or not .

SL is widely used for visual inspection tools across a variety of industries such as identifying soldering defects in circuit boards, micro-cracks in industrial bearings, or analyse genetic markers and medical imagery to predict the trajectory of mental disorder symptoms or treatment outcomes.

SL is limited by it's need for large high-quality datasets to train on as its time consuming to

manually label all data. In addition, novelty can be a problem when deploying an SL system because it can only act based on what it has seen before. If something new was to appear then it couldn't make an effective action (Taye 2023).

Unsupervised Learning

Unsupervised learning (UL) identifies trends and patterns in unlabelled data which may not have been noticed by humans. Groupings and associations are identified in the data which can be useful for analysing data without any prior knowledge of the structure (Ragab et al. 2019). This identification can broadly be split into 3 different categories: clustering, dimensionality reduction, and association rule learning.

Clustering autonomously organises data into groups which have more similarities than data in other groups. This is used in manufacturing, industrial, and cyber security to identify anomalies by detecting differences from normal activity where SL would struggle with unknown actions.

Dimensional reduction compresses high dimensional data into fewer dimensions, preserving the most salient information to reduce noise and computational load. This can most easily be thought of as reducing a picture to it's most basic characteristics such as a face being described by the eyes, nose, mouth, and hair. Using principal component analysis a 10,000 pixel image can be reduced to 50-100 and multiple indicators used to trade stock can be reduced to a few underlying factors.

Association rule learning identifies dependencies and relationships between data points linking them by their occurrence together or following one another. This can be most easily seen in online shops recommending you a certain product after looking at or purchasing a different item (Mhatre n.d.).

As labels aren't predefined there's no explicit right or wrong. Consequently, the accuracy can be hard to determine and results difficult to interpret.

SL has been used to assist UL for fine tuning. Once the UL algorithms has identified generic features SL can create more robust representations of the data to refine results (Taye 2023).

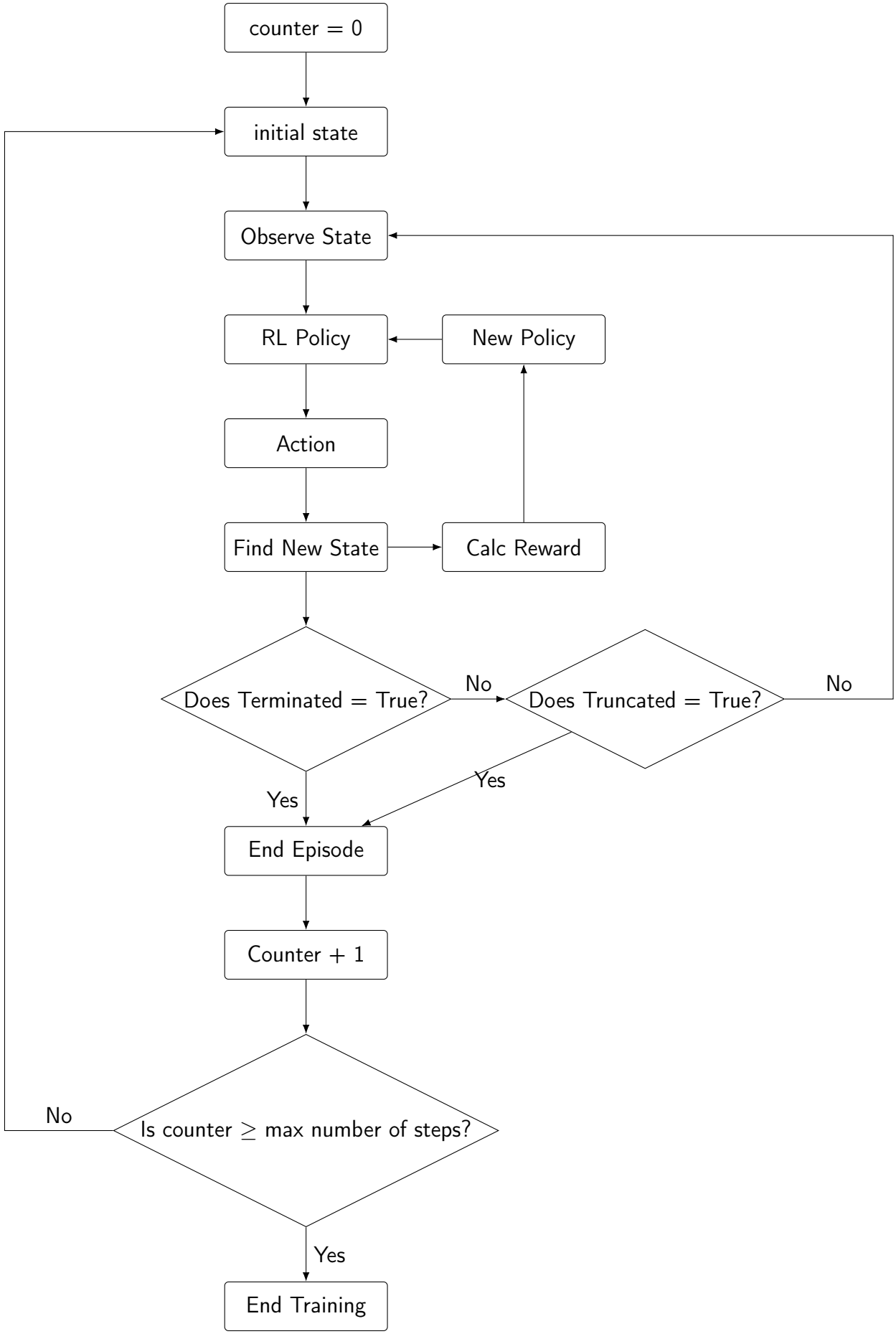
Reinforce Learning

Reinforcement learning (RL) uses a reward mechanism to train models based on them taking actions to achieve goals. This is done by running simulations and getting the model to take actions based on the state of the simulated environment. The model receives a reward as

feedback based on the outcome of the actions which are used to learn a policy based on maximising cumulative rewards. The policy acts as a function taking in an observed state and outputting an action. As shown bellow this process continues until it completes the given task or reaches the maximum number of steps set and is terminated to stop the potential of never completing a task (Taye 2023).

RL is used for complex tasks that require decisions to be made such as driving autonomous vehicles and managing warehouse inventory.

Due to the time consuming nature of running simulations, RL is computationally expensive. This means that to create large quality models expensive hardware is used. In addition, RL is heavily dependant on the reward structure to shape the actions taken. It's critical to set up the rewards for what needs to be accomplished otherwise the policy will be misaligned with the overall objective and fail to generate high qualitiy solutions. Misalignment may also result in the agent exploiting and prioritising subgoals rather than the overall objective (Sutton & Barto n.d.).


counter = 0
initial state
Observe State
RL Policy
New Policy
Action
Find New State
Calc Reward
Does Terminated = True?
No
Does Truncated = True?
No
Yes
Yes
End Episode
Counter + 1
No
Is counter ≥ max number of steps?
Yes
End Training

3.4.2 Use of Machine Learning in Optimisation

Generative Design

Klar et al. (2021) used reinforcement learning to plan factory layouts using a Double Deep Q Learning (DDQL). They modelled the factory floor as a grid and trained an agent to select the rotation and position of a Functional Unit (FU) along an automated guided vehicle (AGV) track. The model learned to plan the layout for reducing transport time and as such placed the FUs close together. This is a good example of optimising before operations even start and an effective solution was found in 8000 episodes.

However, the scalability and use of this was quite limited since only 4 FUs were trained for and doesn't appear to be able to be used on alternative factories with different FUs. In addition, only time was optimised for again limiting the effectiveness in the real world along with the lack of restrictions such as electricity and water supply and usage.

DDQL is usable for such a small scale optimisation but it takes long training times to provide effective results. For use with larger states which would be the case with more FUs and objectives DDQL would struggle to provide effective solutions or cost a lot to train (Sabbir M 2024).

Optimising Production Schedules

Gao et al. (2023) used reinforcement learning to aid PSO by adding it into the algorithm's optimisation process to change the weightings of different components: inertia, individual best position, or global best position. Based on the state of the particles the algorithm can then move between global exploration, local search, slow convergence, and fast convergence. The fitness functions of different iterations are compared to provide a reward function from which a Q-table is updated to be used for future decisions. The iterations are carried out until a termination criteria is met such as a maximum number of iterations or an acceptable error is reached. The logic for this process and the integration of RL into the algorithm can be seen bellow in Figure 3.3

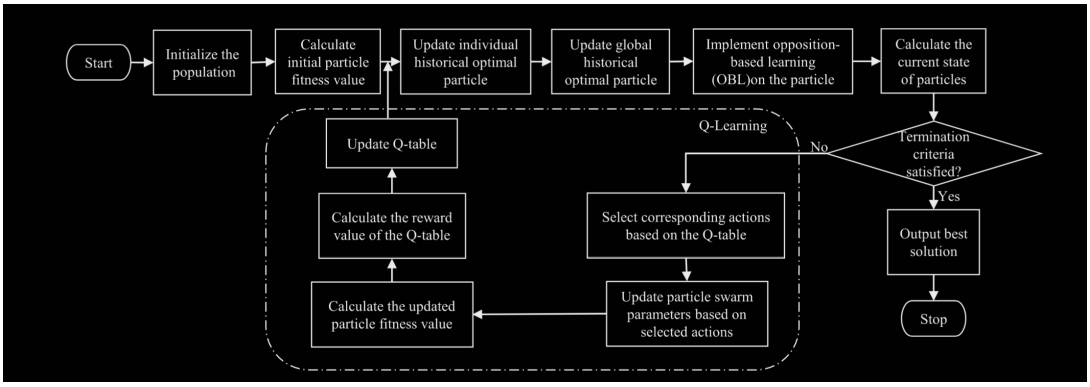


Figure 3.3: The process of improved particle swarm optimization algorithm combined with reinforcement learning

Prediction in Monte Carlo

Li et al. (2025) aims to reduce the computational cost of Monte Carlo while keeping the reliability of results, using a prediction model as a flexible control variate. Conventionally, when running Monte Carlo simulations, a control variate would be used to reduce the variance of simulations by bringing the results closer to a known value. However, in a lot of cases, there are few known control variables, and as such they can't be reliably used. PEMC uses an ML model that is pre-trained to predict control variables for different situations, which can then be used in the MC simulations. This means they only need a few expensive simulations paired with lots of inexpensive simulations. While it's still computationally expensive to train the models and test during deployment and evaluation, results are produced a lot faster than simply running conventional MC simulations.

PEMC was demonstrated to be used across different sectors, such as pricing different financial services and ambulance decision policies. The model performs consistently, reducing estimation barriers compared with standard Monte Carlo methods while providing reliable results. However, this approach doesn't easily generalise across different domains and applications because of the requirements of selecting features that the ML model is specifically trained on. To select these features, significant domain expertise is required and without doing so correctly, the results will be poor. ML models are still at risk of over-fitting, distribution shifts, and model misspecification, which need to be addressed to improve robustness.

3.5 Conclusion

JSSP models the real world challenge of deciding what job should be worked on which machines and when, with different variations, to model characteristics of real-world operations. The literature reviewed demonstrates that the JSSP is a complex and computationally demanding challenge in modern manufacturing, particularly in dynamic environments where frequent state changes require rapid adaptation. Conventional optimisation methods such as heuristics, Genetic Algorithms, and Particle Swarm Optimisation are effective in certain contexts but are often limited by their static nature and the need for repeated re-optimisation. In contrast, ML approaches, particularly RL, offer the potential to improve adaptability and decision-making by learning from system interactions over time. However, practical implementation and evaluation of these methods in realistic scheduling scenarios remain areas requiring further investigation. Therefore, this project explores the use of RL to develop and test an adaptive scheduling system within a simulated manufacturing environment.